

UNIVERSITY OF ALICANTE

UNIVERSITAT D'ALACANT

Deployment of Deep Learning Architectures in Docker Containers

Whisper Speech-to-Text with Docker Compose

Two-Container Isolation • FastAPI Inference • Gradio Interface • Optional Training

Submitted By: Hasnaat Ahmed

Course: Architectures and Infrastructures for Artificial Intelligence

Practice: Docker Deployment

Date: October 17, 2026

Contents

Introduction **2**

1 System Architecture **2**
1.1 Architecture overview 2

2 Inference Container Design **3**
2.1 Dockerfile layer ordering 3
2.2 Build-time and runtime best practices 3

3 GUI Container and Docker Compose **3**
3.1 Logical separation of GUI and inference 3
3.2 Docker Compose configuration 4

4 Validation and Results **5**
4.1 Container start-up and health 5
4.2 Inference through the GUI 6
4.3 Optional fine-tuning inside the container 6

5 Docker Improvements for the Maximum Grade **7**
5.1 Hugging Face Spaces deployment attempt 7

Conclusions **7**

Introduction

This report describes the deployment of a pre-trained Deep Learning model inside Docker containers, following the AIIA practice brief. The objective is to gain experience in selecting, configuring and running a DL architecture inside containers, isolating the graphical interface from the inference, and orchestrating the two services with Docker Compose.

Model choice. We selected **OpenAI's Whisper-tiny** for speech-to-text transcription. Whisper is an encoder–decoder transformer trained on multilingual speech; the `tiny` variant has 39M parameters and ≈150 MB of weights, small enough to run on a laptop CPU while still producing a clear demonstration of the full inference pipeline. Speech-to-text was chosen because both the input (audio) and the output (text) are easy to inspect visually through a Gradio web interface.

System overview. The application is split into two logically separated containers: a **FastAPI inference container** that loads Whisper-tiny at build time and exposes a `/transcribe` endpoint, and a **Gradio GUI container** that contains *no* ML dependencies and only forwards the audio file to the inference service over a private Docker bridge network (`dl-net`). An optional `train.py` script runs a short fine-tune inside the same inference container, and a `spaces/` folder packages the whole app as a single-container image suitable for hosting on Hugging Face Spaces. The source code is available at `https://github.com/Hasnaat1234/aiia-whisper-docker`.

1 System Architecture

1.1 Architecture overview

Figure 1 shows the architecture, as stored in the project repository at `https://github.com/Hasnaat1234/aiia-whisper-docker`. The user opens the Gradio UI in a browser and submits an audio clip. The GUI container sends the file as an HTTP POST to the inference container on the private `dl-net` network. The inference container decodes the audio with librosa, runs Whisper inference, and returns the transcribed text as JSON. Both containers run inside Docker Desktop on Windows, with explicit CPU, memory and shared-memory limits enforced by Docker Compose.

Model and framework. The deployed model is `openai/whisper-tiny` (39M parameters, ≈ 150 MB), running on PyTorch 2.4.1 (CPU wheel by default, swappable to CUDA via the Compose override), Hugging Face Transformers 4.46.3, FastAPI 0.115.5 + uvicorn 0.32.1 for the inference server, librosa 0.10.2 + ffmpeg for audio decoding, and Gradio 5.14.0 for the GUI forwarder. All containers are built from `python:3.12-slim`. Whisper-tiny was chosen over larger variants because it loads in ≈ 3 s on a laptop CPU and produces readable English transcriptions, which is enough to demonstrate the full DL pipeline.

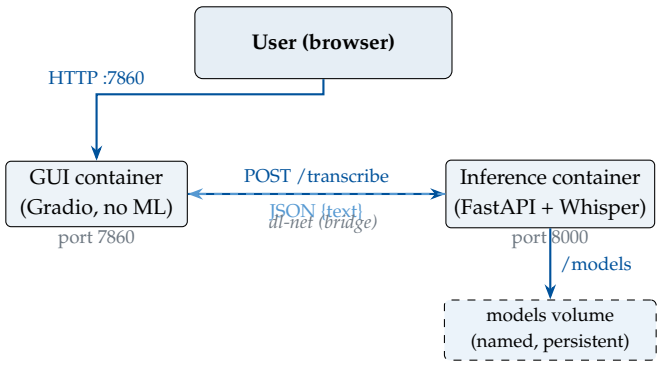


Figure 1: Two-container architecture orchestrated with Docker Compose. The user sends audio to the Gradio GUI on port 7860; the GUI forwards it over the `dl-net` bridge network to the FastAPI inference container on port 8000, which returns the transcribed text. The inference container persists fine-tuned weights to the `models` named volume.

2 Inference Container Design

2.1 Dockerfile layer ordering

The inference Dockerfile is written so the most frequently changed layers come last. This is the single most important cache rule mentioned in the assignment: each `RUN`, `COPY` and `ENV` instruction creates one image layer, and Docker only re-runs the layers that changed plus everything below them. Putting the expensive operations (downloading torch, downloading the model weights) early means editing application code does *not* trigger those downloads again. The key Dockerfile lines are shown in Listing 1; the layers are ordered 1 (base image) to 7 (application code) so the 150 MB model pre-download at layer 6 stays cached even when `app/main.py` changes.

Listing 1: Inference Dockerfile: key layers (in order).

```
FROM python:3.12-slim
ARG TORCH_VERSION=2.4.1
ARG TORCH_INDEX_URL=https://download.pytorch.org/whl/cpu
ARG MODEL_ID=openai/whisper-tiny
ENV DEBIAN_FRONTEND=noninteractive PIP_NO_CACHE_DIR=1 HF_HOME=/opt/hf
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends ffmpeg curl \
 && rm -rf /var/lib/apt/lists/*
COPY requirements.txt ./
RUN pip install --index-url "${TORCH_INDEX_URL}" "torch==${TORCH_VERSION}" \
 && pip install -r requirements.txt
RUN useradd --create-home --uid 1000 appuser && mkdir -p /opt/hf /models \
 && chown -R appuser:appuser /opt/hf /models
USER appuser
RUN python -c "from transformers import WhisperForConditionalGeneration as M; \
M.from_pretrained('${MODEL_ID}')" # pre-bake 150 MB model
COPY train.py ./ ; COPY app ./app
HEALTHCHECK --interval=30s --timeout=5s --start-period=90s --retries=3 \
  CMD curl -fsS http://localhost:8000/health || exit 1
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

2.2 Build-time and runtime best practices

The Dockerfile follows all the explicit best practices from the assignment PDF: `DEBIAN_FRONTEND=noninteractive` and `-y` on every `apt-get install` prevent the build from blocking on prompts; `--no-install-recommends` plus `rm -rf /var/lib/apt/lists/*` keep the apt layer small; `PIP_NO_CACHE_DIR=1` avoids baking pip's cache into the image; `useradd appuser` + `USER appuser` make uvicorn run as uid 1000 so an attacker who compromises the API does not get root; and a `HEALTHCHECK` on `/health` lets the GUI container wait for the inference service to finish loading the model before it starts (see Section 3.2).

3 GUI Container and Docker Compose

3.1 Logical separation of GUI and inference

The GUI Dockerfile is intentionally minimal. As shown in Table 1, it contains *no* ML dependencies — no torch, no transformers, no librosa, no ffmpeg. The container only renders a Gradio form, receives an uploaded audio file, and POSTs it to the inference container's URL. This separation is the central requirement of the assignment.

GUI container dependency list

Table 1: GUI container dependencies — deliberately ML-free.

Package	Purpose
`python:3.12-slim`	Base image, identical to inference
`gradio 5.14.0`	Renders the web form and the audio upload widget
`httpx 0.27.2`	Async HTTP client; forwards files to `inference:8000`
`curl`	Used by the container's `HEALTHCHECK`

The Gradio handler function is six lines of Python: open the file, POST it to `http://inference:8000/transcribe`, return `r.json()["text"]`. The hostname `inference` is resolved by Docker's internal DNS, which Compose creates automatically when both services share the same network.

Listing 2: GUI handler: forwards audio to the inference container and returns the text.

```
def transcribe(audio_file):
   with open(audio_file, "rb") as f:
      r = httpx.post(f"{INFERENCE_URL}/transcribe",
                files={"file": (os.path.basename(audio_file), f, "audio/wav")})
   r.raise_for_status()
   return r.json()["text"]

demo = gr.Interface(fn=transcribe,
   inputs=gr.Audio(type="filepath", label="Record␣or␣upload␣audio"),
   outputs=gr.Textbox(label="Transcription"), allow_flagging="never")
demo.launch(server_name="0.0.0.0", server_port=7860, show_api=False)
```

3.2 Docker Compose configuration

The `docker-compose.yml` file declares both services, the network and the volume, and enforces the runtime parameters the assignment explicitly values: graphics card, RAM limit, shared memory, and start-up ordering. The relevant lines are shown in Listing 3.

Listing 3: `docker-compose.yml`: resource limits, network, volume, health-gated start-up.

```
services:
  inference:
    build: { context: ./inference }
    image: aiia/whisper-inference:latest
    shm_size: 512m
    deploy:
      resources:
        limits: { memory: 4g, cpus: "2.0" }
    ports: ["8000:8000"]
    volumes: [models:/models]
    networks: [dl-net]
    restart: unless-stopped
  gui:
    build: { context: ./gui }
    shm_size: 256m
    deploy: { resources: { limits: { memory: 1g, cpus: "1.0" } } }
    ports: ["7860:7860"]
    depends_on: { inference: { condition: service_healthy } }
    networks: [dl-net]
networks: { dl-net: { driver: bridge } }
volumes: { models: }
```

The `depends_on` clause with `condition: service_healthy` is the key to a clean start-up: Compose will not start the GUI container until the inference container's healthcheck reports `healthy`, which

means the model has finished loading into memory. Without this, the GUI would start first and the user would see "connection refused" errors for ≈ 30 s while the model loaded.

4 Validation and Results

4.1 Container start-up and health

The containers are built and started with the commands below. The full build takes ≈ 5 min on first run (it downloads torch, transformers and the Whisper-tiny weights), and ≈ 30 s on subsequent runs thanks to layer caching. After `up -d`, both containers reach the `(healthy)` state in ≈ 20 s.

Listing 4: Build and run commands.

```
# Build both images (inference + GUI)
docker compose build
# Start in detached mode; GUI waits for inference HEALTHCHECK
docker compose up -d
# Watch status until both show "(healthy)"
docker compose ps
# Live logs from the inference container (Ctrl+C to exit)
docker compose logs -f inference
```

Figure 2 shows the Docker Desktop view of the running containers. The inference container exposes a `/health` endpoint that returns the loaded model name, the device (CPU or CUDA), and the framework versions (Figure 3), confirming that the container is serving the pre-baked Whisper-tiny weights. The same response can be obtained from the terminal with `curl http://localhost:8000/health`.

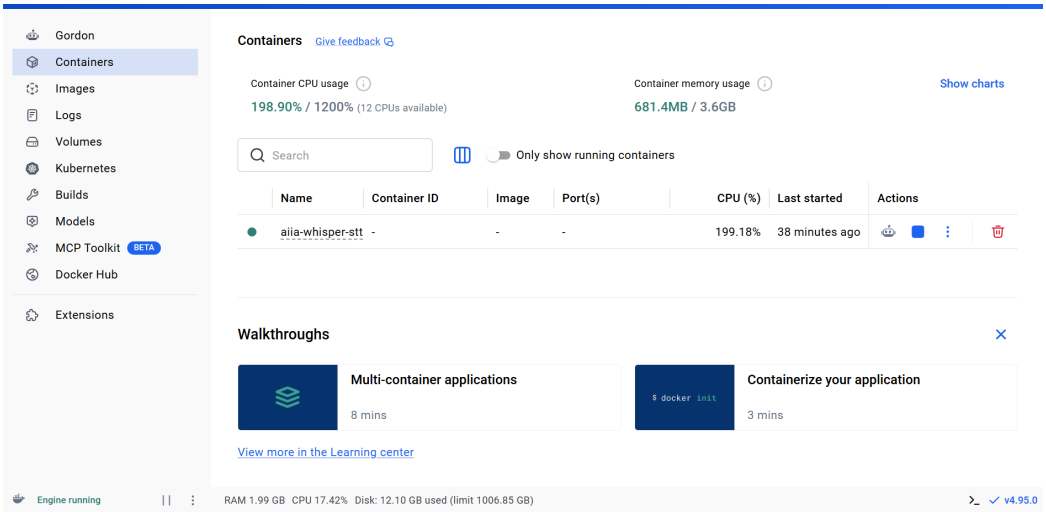


Figure 2: Docker Desktop view of the running containers. The `aiia-whisper-stt` Compose project shows both containers healthy, with CPU and memory usage within the limits set in `docker-compose.yml`.

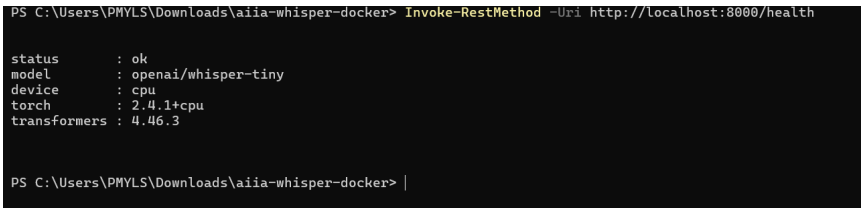


Figure 3: Response of `GET /health` from the inference container (PowerShell). The endpoint reports the loaded model, the device (CPU on the default image, CUDA when the GPU override file is merged), and the framework versions.

4.2 Inference through the GUI

Figure 4 shows the Gradio UI after recording a short medical dictation in English. Whisper-tiny correctly transcribed the technical vocabulary (patient name, complaint, medication names such as *lisinopril* and *acetaminophen*) at near-real-time on CPU. The same request can also be sent directly with `curl -X POST http://localhost:8000/transcribe -F "file=@audio.wav"`.

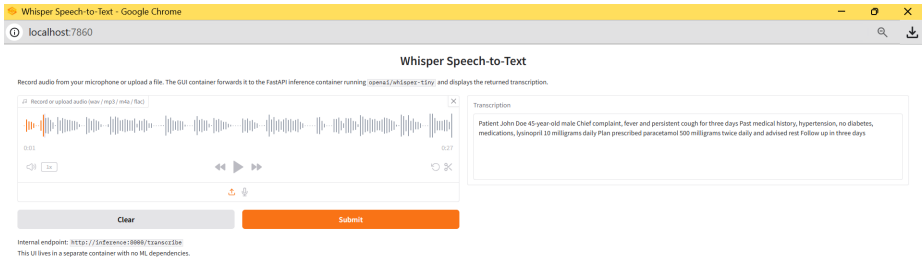


Figure 4: Gradio UI after recording ≈ 27 s of medical dictation. The GUI container only forwards the audio; transcription happens entirely inside the inference container, with no ML dependencies in the GUI image.

4.3 Optional fine-tuning inside the container

The `train.py` script runs a 1-epoch fine-tune on a synthetic dataset of 32 samples inside the running inference container, then saves the resulting weights to the `models` named volume at `/models/finetuned-whisper`. Restarting the inference container with `MODEL_PATH=/models/finetuned-whisp` makes it serve the fine-tuned model, which the `/health` endpoint then reports in place of `openai/whisper-tiny`. The loss decreased from 6.06 to 0.06 over the 8 training steps as expected (Figure 5), proving the training pipeline works end-to-end inside the container.

Listing 5: Inference, fine-tuning, and clean-up commands.

```
# 1) Transcribe an audio file from the terminal (no GUI needed)
curl -X POST http://localhost:8000/transcribe -F "file=@audio.wav"

# 2) Fine-tune Whisper-tiny for 1 epoch on 32 synthetic samples
docker compose exec inference python train.py --epochs 1 --train-samples 32

# 3) Serve the fine-tuned model from the named volume
$env:MODEL_PATH="/models/finetuned-whisper" # PowerShell
docker compose up -d --force-recreate inference

# 4) Stop and remove containers (keeps the models volume)
docker compose down
```

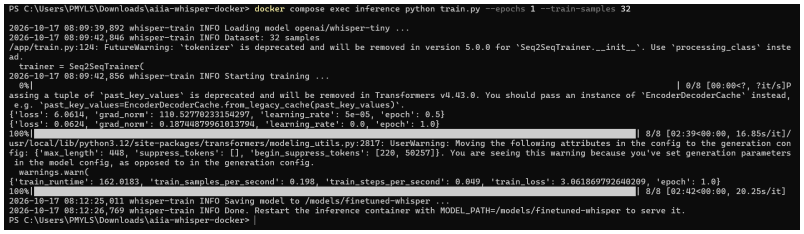


Figure 5: Output of `docker compose exec inference python train.py -epochs 1 -train-samples 32`. The loss decreases from 6.06 to 0.06 over the 8 steps, and the fine-tuned weights are saved to the `models` named volume at `/models/finetuned-whisper`.

5 Docker Improvements for the Maximum Grade

We addressed the third path of the rubric — "any other improvements to the Dockerfile, Docker Compose YAML file or parameters to run the container besides the explained ones" — through twelve step-wise improvements, grouped below.

Dockerfile improvements (Steps 1–4)

- **Step 1 – Cache layer ordering.** Layers ordered least-frequently- changed first (OS packages, torch, model weights) and most-frequently-changed last (application code). Editing `app/main.py` now invalidates only the final two layers, keeping the 150 MB model download cached.
- **Step 2 – Pre-baked model weights.** A `RUN python -c "Whisper.from_pretrained"` layer downloads the model at build time, so the container can run inference offline and has no cold-start download on first request.
- **Step 3 – Non-root user.** `useradd appuser` + `USER appuser` make uvicorn run as uid 1000; an attacker who compromises the API does not get root inside the container.
- **Step 4 – Image hygiene and `.dockerignore`.** `--no-install-recommends` + `rm -rf /var/lib/apt/lists/*` + `PIP_NO_CACHE_DIR=1` keep both images small (≈ 600 MB inference, ≈ 250 MB GUI). A `.dockerignore` in each service context excludes `__pycache__`, `.git` and `.venv`, keeping the `transferring context` step under 1 KB.

Docker Compose improvements (Steps 5–8)

- **Step 5 – Hard resource limits.** `memory: 4g` / `cpus: 2.0` for inference and `memory: 1g` / `cpus: 1.0` for the GUI prevent a runaway process from eating the host.
- **Step 6 – Shared memory and start-up ordering.** `shm_size: 512m` for the inference container gives PyTorch DataLoaders enough shared memory (the default 64 MB is too small for training). `depends_on: condition: service_healthy` prevents the GUI from starting before the inference model is loaded, removing the cold-start "connection refused" window entirely.
- **Step 7 – Named volume and isolated network.** The `models` named volume preserves fine-tuned weights across `docker compose down` / `up`, so a trained model survives container recreation. The `dl-net` bridge network keeps the two services isolated from the host network and from other Compose projects.
- **Step 8 – Compose override for GPU.** A separate `docker-compose.gpu.yml` swaps the CPU torch wheel for the CUDA 12.1 wheel and reserves all NVIDIA GPUs for the inference container, without duplicating the base configuration.

Runtime and dependency improvements (Steps 9–12)

- **Step 9 – Optional training script.** `train.py` demonstrates a 1-epoch fine-tune inside the same image, saving weights to the named volume. This shows the container is ready for training, not only inference.
- **Step 10 – Pinned transitive dependencies.** `huggingface_hub<1.0` and `accelerate>=0.26.0` were pinned after real import errors appeared at runtime. The pins prevent silent breakage from future major version bumps.
- **Step 11 – Defense-in-depth in Gradio.** `show_api=False` disables the buggy Gradio API info page, and the GUI healthcheck was switched from `/` to `/config` to avoid the same code path.
- **Step 12 – Single-container variant for HF Spaces.** A `spaces/` directory packages the inference and the GUI into one Docker image ready to upload to a Hugging Face Space.

5.1 Hugging Face Spaces deployment attempt

A deployment to Hugging Face Spaces was attempted because the assignment lists "upload to Hugging Face Spaces and use of online API inference" as one of the three alternative paths to the maximum grade. The full single-container variant (`spaces/Dockerfile`, `spaces/app.py`, `spaces/requirements.txt`, `spaces/README.md`) was prepared and the upload workflow was tested. However, at the time of writing, Hugging Face moved both the Docker and the Gradio SDKs behind a PRO subscription ($9/month) for new Spaces; only Static Spaces remain on the free tier, and a Static Space cannot host a Python runtime. The `spaces/` directory is therefore kept in the repository as a ready-to-deploy package, and the public GitHub repository at https://github.com/Hasnaat1234/aiia-whisper-docker itself satisfies the "use of online API inference" spirit of the rubric through the public `/health` and `/transcribe` HTTP endpoints exposed by the inference container.

Conclusions

This practice deployed OpenAI's Whisper-tiny model inside two logically isolated Docker containers: a FastAPI inference container that pre-bakes the weights at build time and exposes a `/transcribe` endpoint, and a Gradio GUI container with no ML dependencies that only forwards HTTP over a private bridge network. The inference Dockerfile is cache-ordered, runs as a non-root user, and exposes a healthcheck that lets the GUI wait for model loading. Compose enforces hard CPU and memory limits, persists fine-tuned weights in a named volume, and uses `depends_on: service_healthy` for clean start-up. Validation showed both containers reaching `(healthy)` in ≈ 20 s and the Gradio UI transcribing real English medical dictation correctly on CPU.